

PORTADA

Análisis y Diseño de Sistemas II

Semana 13 — Patrones GoF Estructurales I: Adapter, Facade, Composite

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Ya saben crear objetos sin acoplarse a la clase concreta.
Hoy: cómo *componerlos* sin duplicar código.**

TRANSICIÓN

De crear objetos a componerlos

En las **semanas 11 y 12** vieron GRASP aplicado en UML y los dos primeros patrones creacionales del catálogo GoF: **Factory Method** y **Abstract Factory**. Ambos resuelven el mismo tipo de pregunta: *¿cómo se crea un objeto sin que el cliente dependa de su clase concreta?*

Hoy cambia la pregunta. La familia **estructural** de GoF no se ocupa de cómo nace un objeto, sino de cómo se **combinan clases y objetos ya existentes** para armar estructuras más grandes — sin que esa combinación duplique código ni vuelva frágil el diseño.

CONCEPTO

Qué resuelven los patrones estructurales

Los **patrones estructurales** son la segunda de las tres familias del catálogo GoF (creacional, **estructural**, de comportamiento — vista en la semana 12, slide de las 3 familias). Responden a una pregunta común: *¿cómo se organizan clases y objetos en una estructura más grande, manteniendo el diseño flexible y sin repetir lógica?*

- Unas veces el problema es una **interfaz que no calza** con lo que el cliente espera.
- Otras veces el problema es un **subsistema con demasiadas piezas** para usarlo cómodamente.
- Otras veces el problema es representar una **jerarquía parte-todo** (un árbol) de forma uniforme.

El catálogo GoF define **7 patrones estructurales** en total. Esta semana se ven **6 de los 7**: hoy, martes, **Adapter, Facade y Composite**; el viernes, **Decorator, Proxy y Bridge**, junto con un laboratorio de implementación.

TRANSICIÓN

Primer problema: una interfaz que no es la que el cliente espera

Imaginen que su dominio ya define cómo quiere hablar con un servicio de pagos: un método `cobrar(monto, moneda)` . Pero el proveedor externo que van a usar (una librería, una API de un tercero, o código legado del mismo proyecto) expone un método completamente distinto, con otros nombres de parámetros y otro formato de datos. El dominio no puede modificar esa librería externa.

CONTRA EJEMPLO

El dominio hablando directamente con la interfaz externa

```
class ProcesadorDeCompras:
    def __init__(self):
        self.pasarela = PasarelaPagosLegacy()

    def pagar(self, monto, moneda):
        # el dominio se dobla para hablar
        # el idioma de la librería externa
        codigo = self.pasarela.ejecutar_transaccion(
            {"amount_cents": int(monto * 100),
             "curr": moneda.upper()})
        return codigo == "OK"
```

`ProcesadorDeCompras` —una clase del dominio— conoce el formato exacto que exige `PasarelaPagosLegacy` (centavos como entero, moneda en mayúsculas, un código de texto como respuesta). Si el proveedor cambia o se reemplaza por otro, hay que tocar el dominio.

CONCEPTO

Definición: Adapter

Adapter (Adaptador) convierte la interfaz de una clase existente —a menudo de una librería externa o de código legado, con una interfaz incompatible— en otra interfaz que el cliente ya espera, **sin modificar el código original** de esa clase.

El adaptador es una clase nueva que **envuelve** (compone) al objeto de interfaz incompatible y traduce cada llamada: recibe la petición en el formato que el dominio espera, y por dentro la reescribe en el formato que la librería externa exige — y viceversa con la respuesta.

CONCEPTO

Adapter, en código

```
interface PasarelaDePagos:          # interfaz esperada
    def cobrar(self, monto, moneda) -> bool: ...

class AdaptadorPasarelaLegacy(PasarelaDePagos):
    def __init__(self):
        self.legacy = PasarelaPagosLegacy()

    def cobrar(self, monto, moneda):
        codigo = self.legacy.ejecutar_transaccion(
            {"amount_cents": int(monto * 100),
             "curr": moneda.upper()})
        return codigo == "OK"
```

Ahora `ProcesadorDeCompras` recibe una `PasarelaDePagos` inyectada (igual que en DIP) y llama `pasarela.cobrar(monto, moneda)`. Ya no conoce `PasarelaPagosLegacy` ni su formato de datos: toda la traducción vive dentro de `AdaptadorPasarelaLegacy`.

CONCEPTO · RELACIÓN CON LO VISTO

Adapter y DIP: el adaptador es la abstracción hecha concreta

En la **semana 9** vieron **DIP (Dependency Inversion Principle)**: los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones. En el ejemplo anterior, `ProcesadorDeCompras` (alto nivel) depende de la interfaz `PasarelaDePagos` (abstracción), no de `PasarelaPagosLegacy` (bajo nivel, de un tercero).

`AdaptadorPasarelaLegacy` es exactamente la pieza que DIP exige que exista en algún lugar: una **implementación concreta** de la abstracción que el dominio ya define. Adapter es el patrón con nombre y estructura fija para el caso particular en que esa implementación concreta debe traducir hacia una interfaz externa que no se puede modificar.

TRANSICIÓN

Segundo problema: un subsistema con demasiadas piezas

Adapter resuelve una interfaz incompatible. Pero hay otro problema distinto: un subsistema que **sí** tiene una interfaz correcta, pero está formado por **varias clases que colaboran entre sí**, y el cliente tendría que conocer y coordinar todas ellas para completar una sola operación de negocio.

CONCEPTO

Definición: Facade

Facade (Fachada) provee una **interfaz simplificada y unificada** a un subsistema complejo compuesto por varias clases y sus interacciones internas. El cliente llama a un único método de la fachada, y esta se encarga de **orquestrar** las llamadas a las clases internas en el orden correcto.

Un punto clave: Facade **no oculta por completo** el subsistema. El cliente puede seguir accediendo directamente a las clases internas si necesita algo puntual que la fachada no expone — simplemente, la mayoría de las veces no hace falta, porque la fachada cubre el caso de uso común.

CONCEPTO

Facade, en código

```
class ValidadorDePedido:
    def validar(self, pedido): ...
class Inventario:
    def reservar(self, pedido): ...
class Pagos:
    def cobrar(self, pedido): ...
class Notificaciones:
    def enviarConfirmacion(self, pedido): ...

class FachadaDePedidos:
    def procesarPedido(self, pedido):
        ValidadorDePedido().validar(pedido)
        Inventario().reservar(pedido)
        Pagos().cobrar(pedido)
        Notificaciones().enviarConfirmacion(pedido)
```

El cliente solo llama `FachadaDePedidos().procesarPedido(pedido)` . Las cuatro clases internas siguen existiendo y siguen siendo utilizables por separado — la fachada solo evita que el cliente tenga que coordinarlas manualmente cada vez.

TENSIÓN CONCEPTUAL

Cuando la Facade absorbe lógica que no es suya

Una Facade bien diseñada **delega**: cada método interno sigue viviendo en su clase (`ValidadorDePedido` , `Inventario` , `Pagos`). El riesgo es que, con el tiempo, alguien empiece a agregar reglas de negocio directamente **dentro** de `procesarPedido()` —validaciones extra, cálculos, condicionales— en vez de en las clases que corresponden.

Ese es el mismo problema que ya vieron con **God Controller / God Object** (semanas 7 y 10): una clase que termina concentrando demasiadas responsabilidades. Una Facade que deja de delegar y empieza a **reemplazar** el subsistema —en vez de solo simplificar su acceso— se convierte en un God Object con otro nombre.

TRANSICIÓN

Tercer problema: tratar "uno" y "muchos" de la misma forma

Adapter traduce una interfaz. Facade simplifica un subsistema. El tercer problema de hoy es distinto: hay estructuras que son naturalmente un **árbol** —una categoría contiene productos y también subcategorías, que a su vez contienen más productos— y el cliente quisiera operar sobre **un elemento suelto** o sobre **una rama completa del árbol** exactamente de la misma manera, sin distinguir casos.

CONCEPTO

Definición: Composite

Composite compone objetos en estructuras de árbol para representar jerarquías **parte-todo**, de modo que los clientes traten un objeto individual (una "hoja") y una composición de objetos (un "nodo" con hijos) **de manera uniforme**, a través de una interfaz común.

Tanto la hoja como el nodo compuesto implementan la **misma interfaz**. La hoja resuelve la operación directamente; el nodo compuesto la resuelve **delegando recursivamente** en cada uno de sus hijos —que a su vez pueden ser hojas u otros nodos compuestos— y combinando los resultados.

CONCEPTO

Composite, en código

```
interface ElementoCatalogo:
    def calcularTotal(self) -> float: ...

class Producto(ElementoCatalogo):    # hoja
    def __init__(self, precio):
        self.precio = precio
    def calcularTotal(self):
        return self.precio

class Categoria(ElementoCatalogo):    # nodo compuesto
    def __init__(self):
        self.elementos = []           # Productos u otras Categorías
    def agregar(self, elemento):
        self.elementos.append(elemento)
    def calcularTotal(self):
        return sum(e.calcularTotal() for e in self.elementos)
```

El cliente llama `calcularTotal()` sobre un `Producto` suelto o sobre una `Categoria` con subcategorías anidadas, sin necesitar saber cuál de los dos es — la recursión de `Categoria` resuelve el árbol completo.

INTERACCIÓN

¿Adapter, Facade o Composite?

En el chat, para cada caso indiquen cuál de los tres patrones calza mejor:

1. Su aplicación necesita usar una librería de generación de PDF de un tercero cuya interfaz no se parece en nada a la interfaz de exportación que ya define el dominio.
2. Un módulo de "cierre de caja" que internamente coordina cinco clases (conteo de ventas, arqueo, impuestos, reporte, respaldo) y el cliente solo quiere invocar `cerrarCaja()` .
3. Un árbol de carpetas y archivos donde se necesita calcular el tamaño total de una carpeta, que puede contener archivos sueltos y otras carpetas anidadas.

5 minutos · respondan en el chat

TENSIÓN CONCEPTUAL

No toda integración externa necesita un Adapter

Si la interfaz de la librería externa ya calza razonablemente bien con lo que el dominio necesita, envolver esa librería en un Adaptador de todas formas agrega una **capa de indirección sin beneficio real**: más código para mantener, sin resolver ninguna incompatibilidad concreta.

El mismo criterio de la semana pasada aplica aquí: un patrón se justifica cuando hay un problema **real** que resolver —en este caso, una incompatibilidad de interfaces genuina—, no porque "envolver todo en una interfaz propia" suene a buena práctica por defecto. Si no hay traducción que hacer, una dependencia directa y bien inyectada (vía DIP) ya es suficiente.



SÍNTESIS

Los tres primeros patrones estructurales, en una frase

1. **Estructurales:** la segunda familia GoF — cómo componer clases y objetos en estructuras más grandes sin duplicar código.
2. **Adapter:** traduce una interfaz incompatible (a menudo externa o legada) hacia la interfaz que el dominio ya espera, sin tocar el código original — suele ser la implementación concreta de una abstracción definida por DIP.
3. **Facade:** una interfaz simplificada que orquesta un subsistema de varias clases, delegando en ellas sin ocultarlas por completo ni absorber su lógica de negocio.
4. **Composite:** una interfaz común para hojas y nodos compuestos permite tratar "uno" y "un árbol de muchos" exactamente igual.
5. Ninguno se justifica sin un problema real detrás: incompatibilidad genuina, subsistema realmente complejo, o una jerarquía parte-todo real.



CIERRE

El viernes: los otros tres patrones estructurales

Hoy vieron 3 de los 6 patrones estructurales que cubre esta semana. El **viernes** se completan los otros tres: **Decorator** (agregar responsabilidades a un objeto dinámicamente, sin herencia), **Proxy** (un sustituto que controla el acceso a otro objeto) y **Bridge** (separar una abstracción de su implementación para que ambas varíen de forma independiente), junto con un **laboratorio de implementación** aplicando los seis patrones estructurales al proyecto de curso.

Recuerden: el miércoles de esta semana es Coworking (Planning Sprint 3) — no hay sesión teórica nueva ese día.

